CLEW

AGENTIC DEVELOPMENT · 2026

The model is the engine. The harness is the car.

What makes a good agentic coding harness — and how clew keeps it coherent.

SECTION 01 / 05

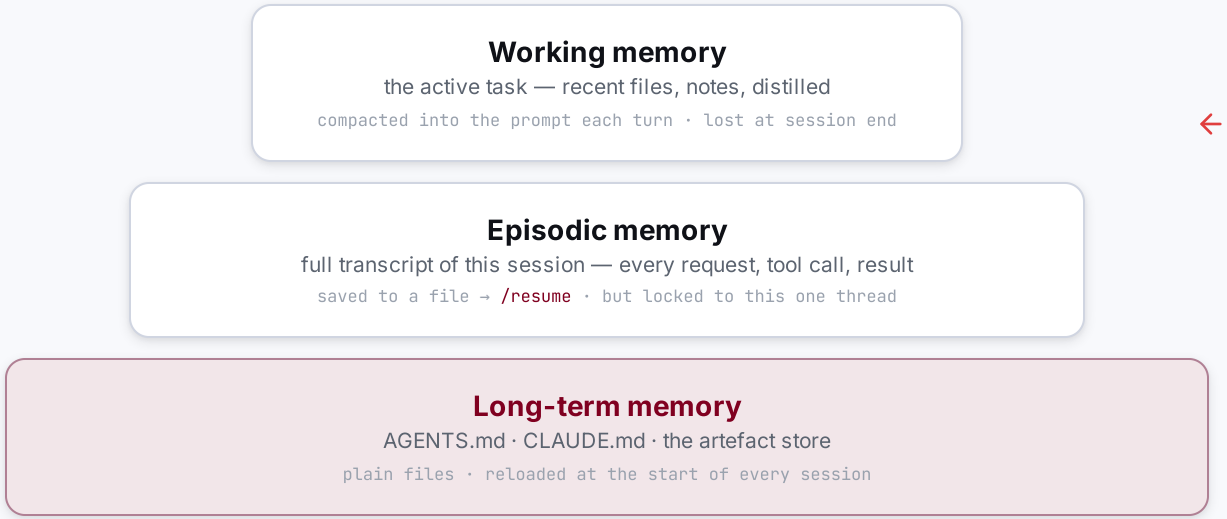
The Problem

Agents can't remember between sessions — and every obvious fix for it fails.

01

THE GAP

Most harnesses only have the shallowest memory — yet memory is the moat.



← most stop here

the moat

2 of 3 session-bound

Working memory is distilled, then discarded at session end; episodic is a resumable transcript but locked to its own thread. Only long-term — plain files — crosses into the next session.

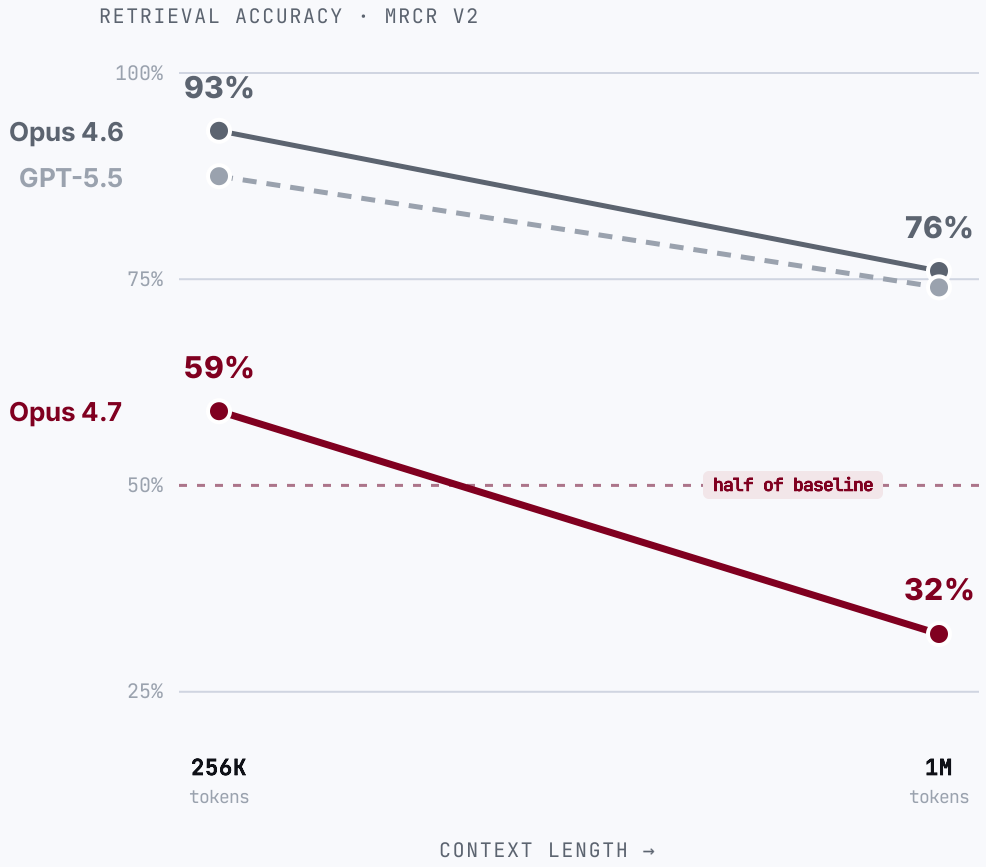
0 tokens / fetch

And it's a local file read — no MCP API roundtrip, no token cost, reloaded at the start of every session.

THE OBVIOUS OBJECTION *"A frontier window holds a million tokens now — so why not just let the window be the memory?"* ANSWERED NEXT →

No — a bigger context window is not bigger memory.

Bury one fact in a million-token prompt and ask for it back: the frontier model returns it barely **1 in 3** times. The window **forgets** as you fill it.



The advertised window is **not** the usable window.

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context — a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens — context rot isn't new (NoLiMa, 2025).

Writing it down isn't memory either.

OpenAI shipped a 1M-line product with **zero** hand-written code — then found the obvious fix, one big context file, fails.

1M lines of code | **0** hand-written | **1,500** PRs merged

"One big `AGENTS.md`" failed four ways

- **Crowds out the task.** Context is scarce — the giant file pushes out the code and the docs.
- **Too much guidance becomes none.** When everything is "important," nothing is.
- **It rots instantly.** A graveyard of stale rules — the agent can't tell what's still true.
- **It can't be verified.** Drift is inevitable without mechanical checks.

If it's not in-context, it doesn't exist

Anything in Slack, Google Docs, or someone's head is invisible to the agent.

OpenAI's fix

A **~100-line table of contents** pointing into a **structured `docs/` tree** — a map, not an encyclopedia.

SECTION 02 / 05

The Agentic Harness

Why the tooling around the model — not the model — is the performance lever.

02

THE LEVER · REALITY CHECK

2025 was the “Year of the Agent.” It didn't land.

The hype was model-led — but in production, the model alone fell short, expensively.

95%

of enterprise GenAI pilots showed no measurable P&L impact

MIT studied 300 deployments + 350 leaders. The blocker wasn't model IQ — it was integrating AI into real workflows.

~59%

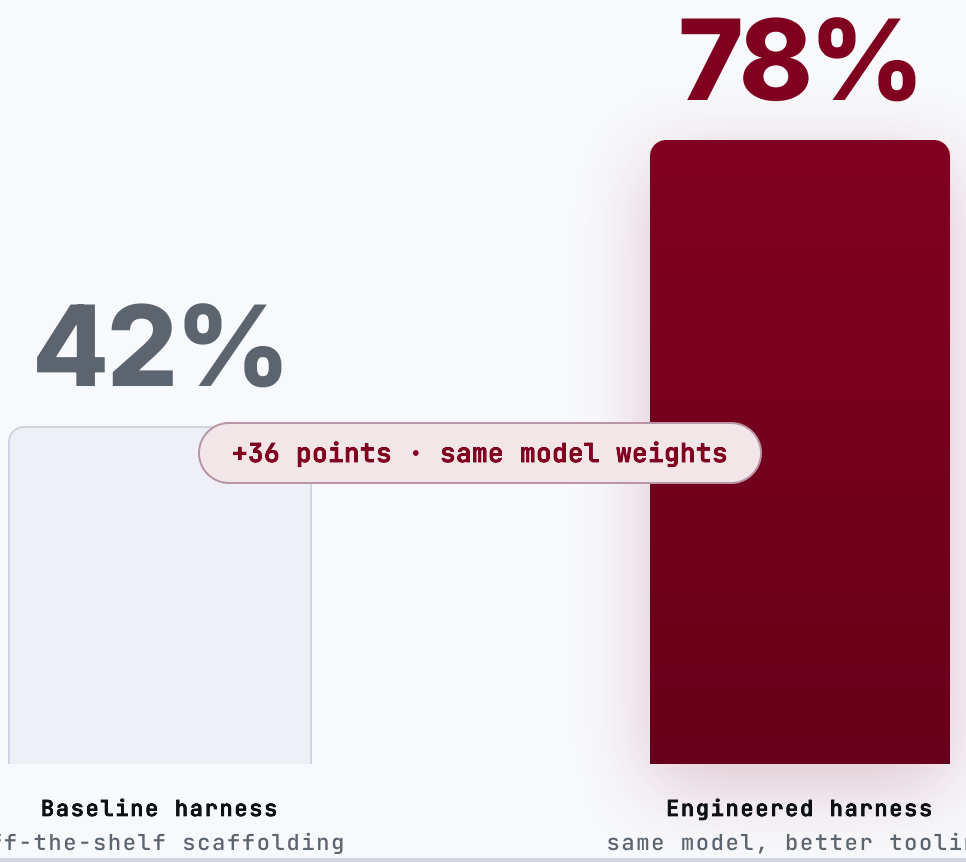
is the best frontier model's score on realistic enterprise tasks

Scale's SWE-bench Pro — 1,865 real tasks across 41 professional repos (2026). The same models score 80–95% on the benchmark everyone quotes; the gap is the reality.

The missing variable wasn't intelligence — it was the **harness around the model**.

MIT Project NANDA · The GenAI Divide: State of AI in Business 2025 (95% of pilots — no measurable P&L impact) · via Fortune (2025) — fortune.com | Scale AI (SEAL) · SWE-bench Pro…

Hold the model fixed, swap the harness: 42% → 78%.

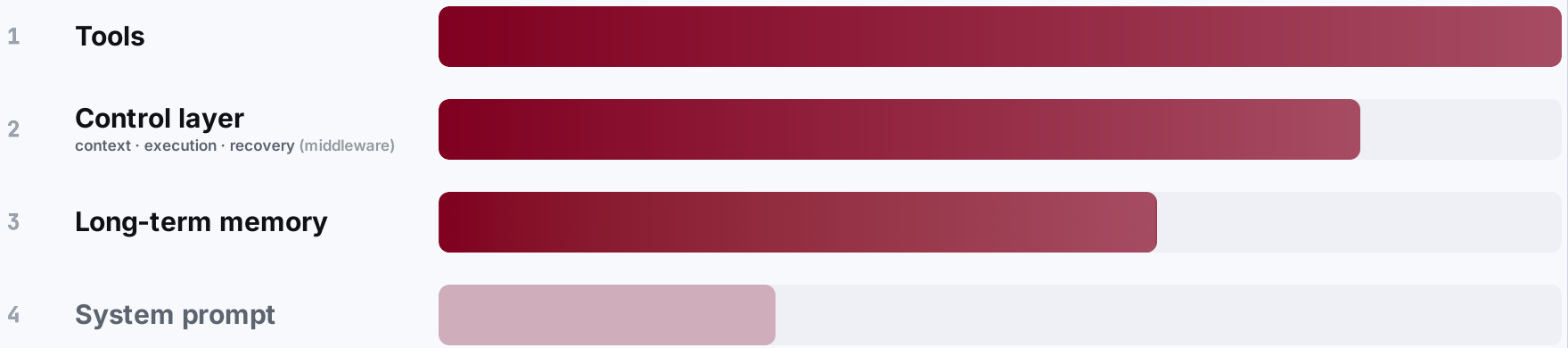


Claude Opus 4.5 · HAL CORE-Bench · the only thing that changed was the harness

THE LEVER · WHERE IT LIVES

Performance lives in the tooling around the model — not the model.

Ablation studies rank what actually moves agent performance. Model prompt-tuning sits **last**.



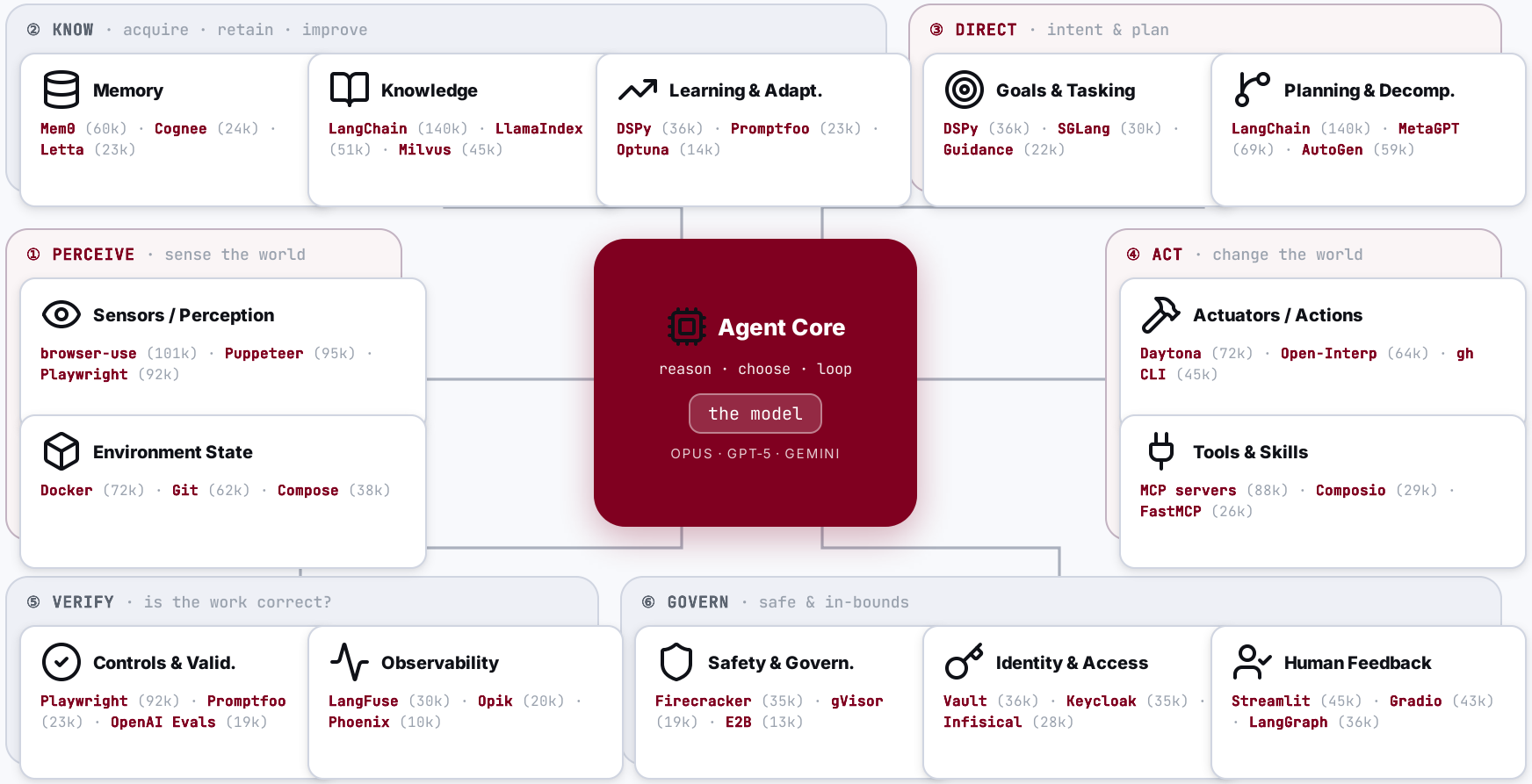
THE INVERSION

Most teams pour effort into #4 — the prompt. The gains sit in #1–3.

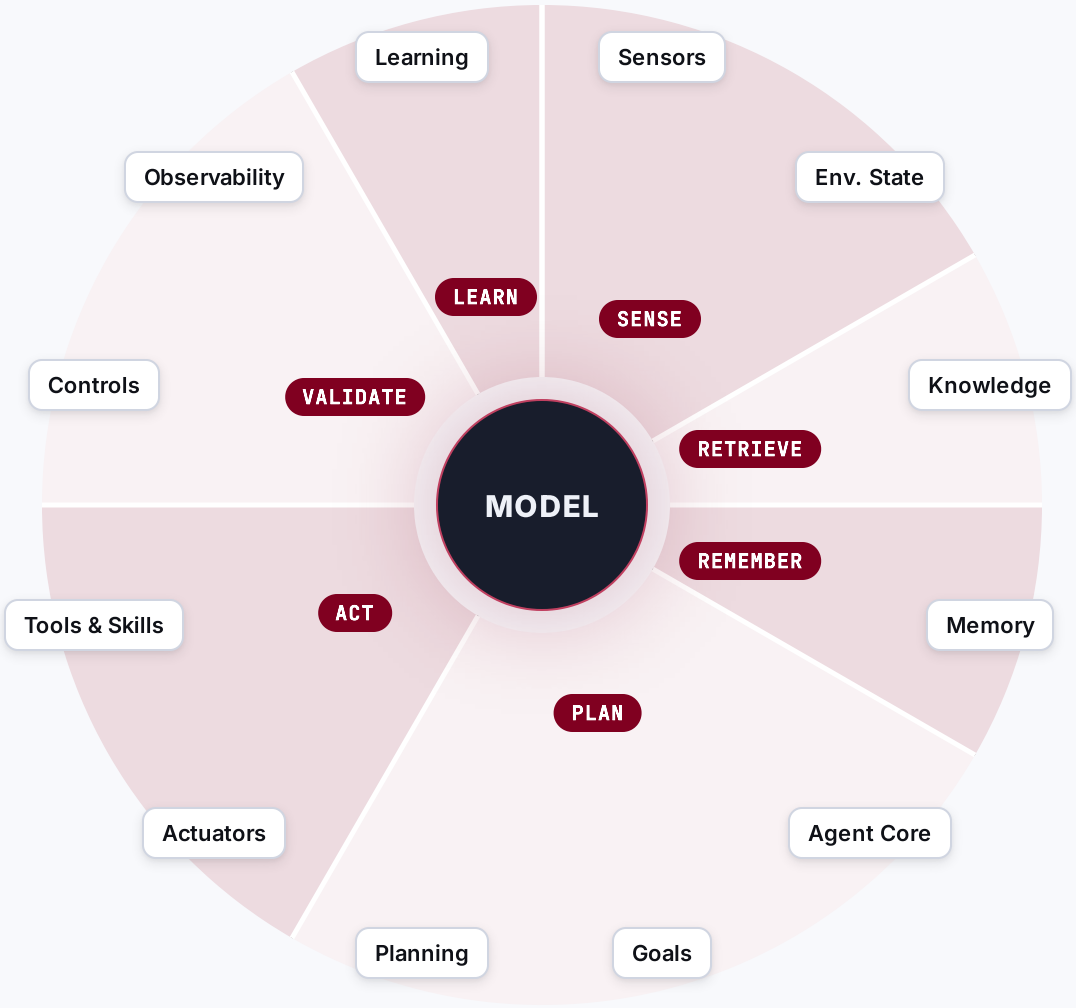
THE MODEL

Table stakes once it is capable. The harness is the differentiator.

The agent core, wrapped in *14 capabilities* — and who leads each.



The model is a dot. The harness is the mass around it.



CROSS-CUTTING

Govern every step — they don't belong to one.

- Safety & Governance
- Identity & Access
- Human Feedback

Each wedge is **one loop step** and the capabilities it owns · the three on the right **govern every step**.

SECTION 03 / 05

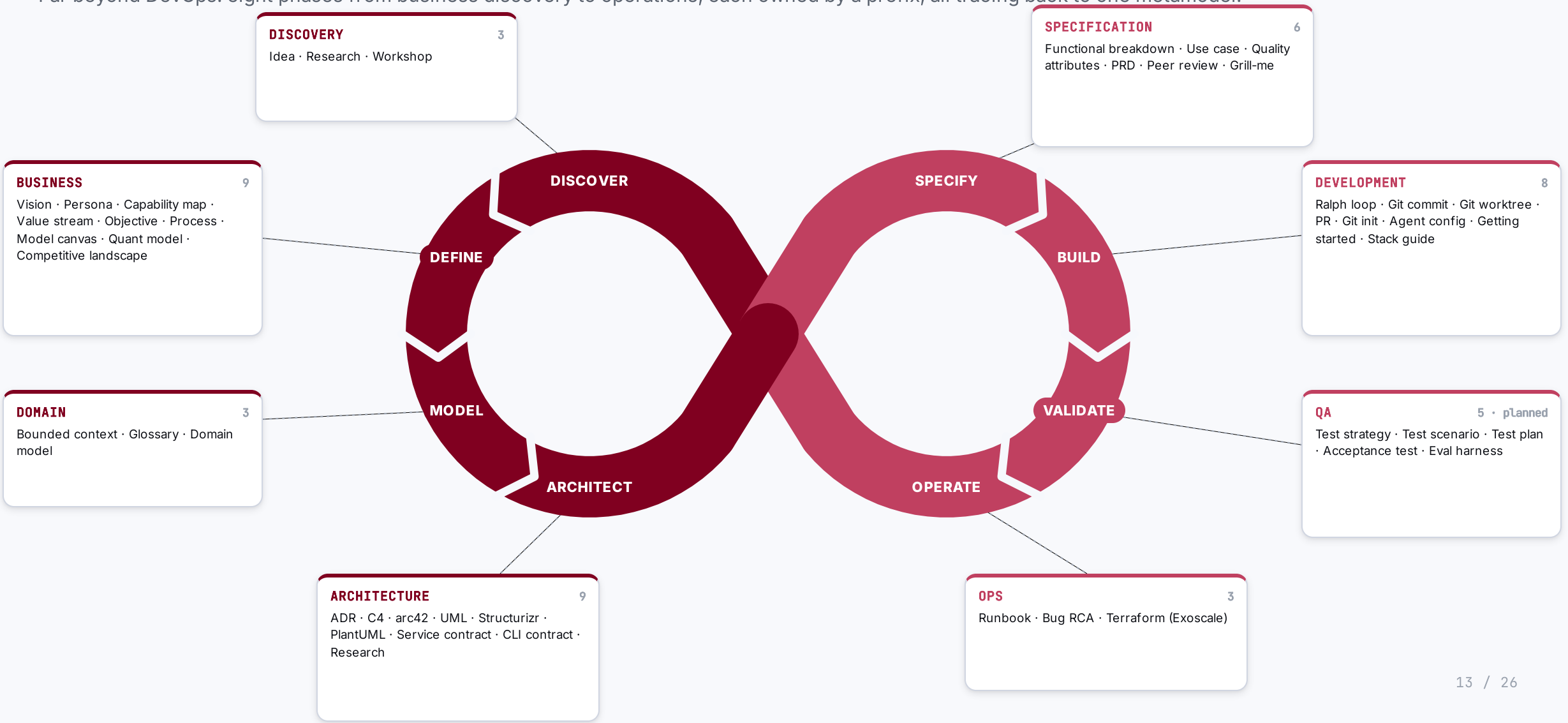
The Metamodel

How an agent armed with skills and rules turns business context into deployable artefacts — every step traceable.

03

Skills & rules wrap the whole lifecycle — from discovery to operations.

Far beyond DevOps: eight phases from business discovery to operations, each owned by a prefix, all tracing back to one metamodel.



Discover & Define — make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Brainstorm & capture** — ideas, hypotheses
- **Research & scan** — market, competitors
- **Interview & workshop** — with the business
- **Model the business** — vision → objectives
- **Define the domain** — ubiquitous language

Business
9 SKILLS

- **Vision** — north star
- **Persona** — who we serve
- **Model canvas** — value model
- **Capability map** — what it can do
- **Competitive landscape** — market position
- **Value stream** — how value flows
- **Objective** — goals & KRs
- **Process** — operations
- **Quant model** — unit economics

Discovery
3 SKILLS

- **Idea** — capture & graduate
- **Research** — 1:1 interviews
- **Workshop** — group facilitation

Domain
3 SKILLS

- **Bounded context** — boundaries
- **Glossary** — ubiquitous language
- **Domain model** — aggregates & events

Architect the solution — turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Decide architecture**
  ADRs, C4, arc42
- **Define contracts**
  service, CLI
- **Break down function**
  FBS
- **Specify & qualify**
  use cases, quality
- **Write & review PRDs**
  one per epic

Architecture
9 SKILLS

- **ADR**
  decisions
- **Research**
  arch notes
- **Service contract**
  API surface
- **CLI contract**
  command surface
- **Structurizr**
  C4 workspace
- **C4**
  context → component
- **PlantUML**
  UML pipeline
- **UML**
  sequence/class/state
- **arc42**
  arch narrative

Specification
6 SKILLS

- **Functional breakdown**
  feature tree
- **Use case**
  actor scenarios
- **Quality attributes**
  ISO 25010
- **PRD**
  one per epic
- **Peer review**
  PRD/plan review
- **Grill-me**
  adversarial review

UX
1 SKILL

- **Design system**
  tokens & brand

Execute & Deliver — plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Sequence the roadmap** — epics, phases
- **Plan increments** — atomic steps
- **Build & validate** — tests, checks
- **Deploy** — infra, release
- **Operate & fix** — runbooks, RCA

Specification
2 SKILLS

- **Delivery roadmap** — epics & phases
- **Implementation plan** — atomic increments

QA
5 · PLANNED

- **Test strategy** — pyramid & coverage
- **Test scenario** — from use cases
- **Test plan** — per release
- **Acceptance test** — Given–When–Then
- **Eval harness** — agent evals

Development
8 SKILLS

- **Agent config** — CLAUDE.md & rules
- **Getting started** — onboarding
- **Stack guide** — tech patterns
- **Git init** — repo setup
- **Git commit** — commits
- **Git worktree** — worktrees
- **PR** — pull requests
- **Ralph loop** — autonomous loop

Ops
3 SKILLS

- **Runbook** — operations
- **Bug RCA** — incident analysis
- **Terraform** — Exoscale infra

fed by ← Design

SECTION 04 / 05

The Build Workflow

From a PRD to merged code — how one engineer drives agents through alignment and verification gates, then lets them run autonomously.

04

THE BUILD WORKFLOW · END TO END

From PRD to merged code — two fresh-session gates, then autonomy.

Business context and discovery are done; the roadmap gives one PRD per epic. The loop below runs per PRD.

1 Spec ready

ALREADY DONE

- Delivery roadmap → **one PRD per epic**
- Each PRD states what must be true

PRD in hand

2 Align & plan

FRESH SESSION

- `agent-grill-me`: agent grills you ~10 Q on impl & architecture
- you answer → shared mental model
- agent writes the **implementation plan**

Plan drafted

3 Verify the plan

FRESH SESSION

- `agent-peer-review`: findings ranked critical → low
- you resolve the ones needing guidance → amend
- `ralph audit`: plan is autonomously runnable

Audit green

4 Execute & ship

SEPARATE CLI

- `agent-ralph-loop ralph.sh`: fresh agent per increment
- implement → test gate → commit, until done
- run CI on the finished work

Merged

Two **fresh-session** checks (grill-me before, peer-review after) keep me and the agent aligned; the loop only runs once the plan is proven autonomous.

PRD says what must be true; the plan says how — safely.

Two artefacts, one chain: the PRD scopes the epic, the implementation plan breaks it into runnable increments.

BUILD BY FEATURE · ONE PER EPIC

PRD

“What must be true, and why.”

- **Problem, goals & non-goals** — the scope boundary
- **User stories** grounded in personas (P-NN)
- **Acceptance criteria** + success metrics
- **§0 traceability** — the epic and its dependencies

E-NN · P-NN · C-N.M.FXX · QA-XXNN · UC-NN

1 PRD
↓
1 plan

acceptance criteria become the plan’s test gates

increments deliver the PRD scope

ATOMIC INCREMENTS · ONE PER PRD

Implementation plan

“How we get there, reversibly.”

- **Ordered increments** (Inc-1 … Inc-N)
- each **small · testable · reversible**
- scope + primary files per increment
- **test gate** + exit criteria = done

Plan-NNNN · Inc-N · Status: pending → done

Two fresh agents pressure-test the work — before and after the plan.

Grill-me pulls the thinking out of my head; peer-review checks the document. Both run in a clean session.

ALIGNMENT · BEFORE THE PLAN

agent-grill-me

Live Socratic session on the PRD — one question at a time.

- Agent asks ~10 focused Q on **implementation & architecture**
- Vocabulary checked against the **domain glossary**
- Decisions crystallised into **ADRs** while context is live
- Surfaces the **thinking gaps** only questioning reveals

Output: **shared mental model** → the agent then writes the implementation plan.

VERIFICATION · AFTER THE PLAN

agent-peer-review

Static, independent scan of the finished plan.

- Reads the plan end-to-end, builds a requirement map
- Findings ranked with concrete remediation:

  critical · major · normal · low

- I resolve the ones needing guidance → **amend the plan**

Output: **validated plan** → ready for the autonomy audit.

Peer-review finds the **mechanical** gaps automatically; grill-me surfaces the **thinking** gaps under questioning.

The Ralph loop — a fresh agent per increment, until the plan is done.

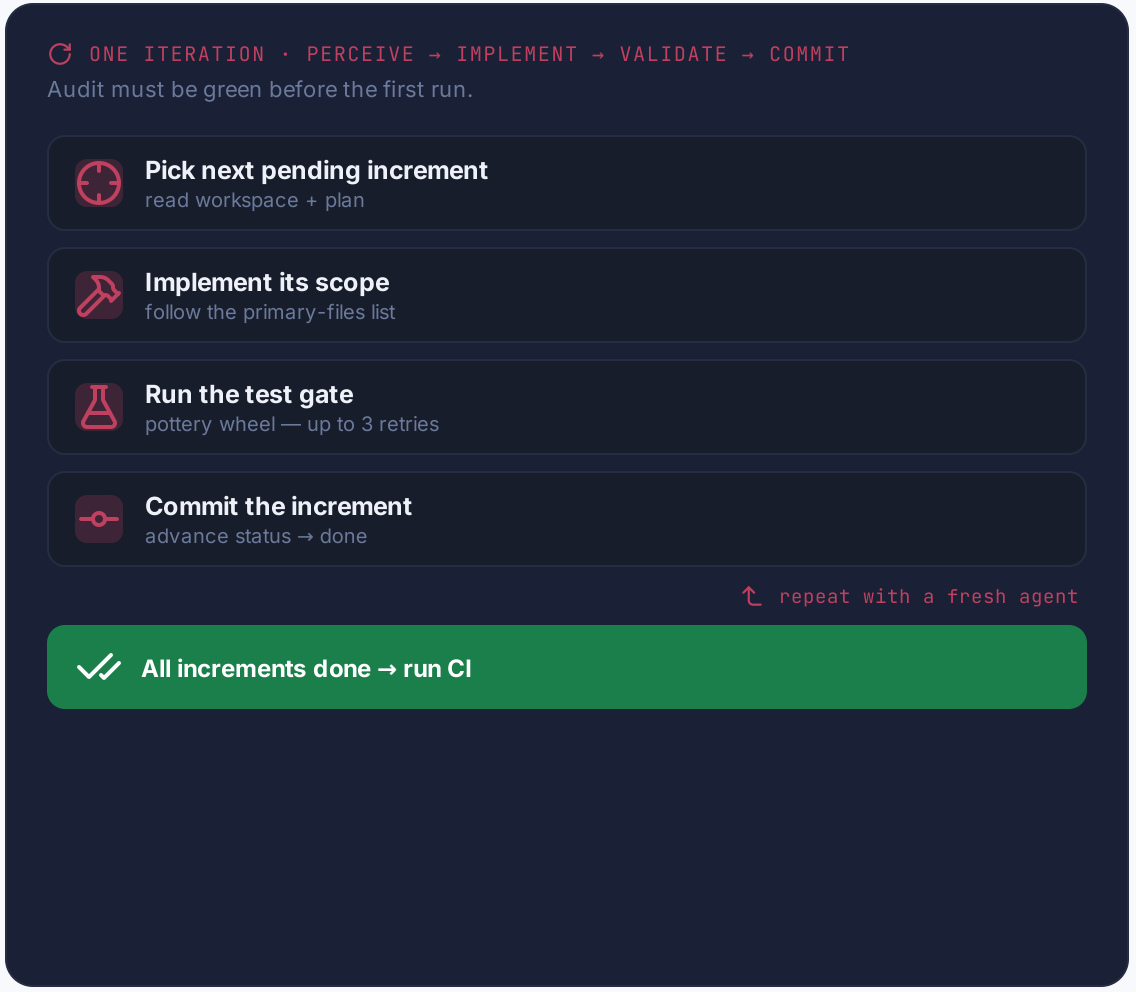
WHAT THE LOOP IS

A bash script (**ralph.sh**) drives the plan to completion — one increment per iteration.

Fresh agent each iteration → clean context, no drift or context rot.

Loops while increments are **pending**; stops when every one is **Status: done**.

```
ralph.sh <workspace> \
  --max-iterations 50 --with-prd --with-push
# spawns: claude --print < iteration-prompt
```



SECTION 05 / 05

clew

The engine that persists the whole graph: deterministic IDs, write-time integrity, and end-to-end traceability.

05

clew engineers the deepest memory layer — so it stops rotting.

WITHOUT STRUCTURE, MEMORY ROTS

No traceability graph from the docs
No reliable view of how artefacts relate — joins across them are impossible.

No determinism if you trust the LLM
It silently renumbers IDs; you end up re-verifying everything by hand.

Refactors cost a fortune
Change one ID or name and the effort to update every reference is massive.

CLEW = STRUCTURED LONG-TERM MEMORY

End-to-end traceability
a typed graph generated from the docs themselves

Write-time integrity
deterministic IDs · FK checks · drift detection

Zero token cost per fetch
local file reads · the repo is the source of truth

HOW IT WORKS

IDs minted by the app, links checked at write time — never by the LLM.

1 · AGENT ASKS

Call clew via Bash

```
clew new prd --epic E-01
```

The agent never invents an ID — it requests one.

2 · CLEW ENFORCES

Validate & mint

```
E-01 exists? ✓
FK enforced · → PRD-0001
```

Type-checked against the relationship registry; collisions rejected.

3 · AGENT WRITES

Reference the ID

```
[PRD-0001](prd-0001.md)
```

Markdown prose cites the stable ID clew returned.

Deterministic — clew mints every ID and checks every link

Guessed — the LLM renumbers silently and references rot

THE PAYOFF

Ask the graph anything: trace upstream, measure blast radius, prove integrity.

clew trace OBJ-02

```
OBJ-02
↑ E-01 Epic
↑ C1.2 Capability
↑ P-01 Persona
↑ VISION
```

Upstream chain — why this exists.

clew impact E-03

```
E-03 changes →
↓ PRD-0007
↓ QA-SC02
↓ Plan-0007
3 artefacts affected
```

Downstream blast radius — what breaks.

clew check

```
scanning references…
bindings ✓
drift ✓
orphans ✓
0 broken references
```

Integrity audit — deterministic, repeatable.

100% current Every internal reference is up to date, **all the time** — the north-star metric is end-to-end traceability, not work-in-progress.

THE TAKEAWAY

Engineer the harness, not just the prompt.

Start with the highest-leverage capability: memory that doesn't rot.

Not a PM tool | Not a wiki | Not a SaaS | The repo is the source of truth

Git for product architecture — manage what must be true, not what's in progress.